

BCA 5th Semester

PHP

Topic : PHP Basics

- By Soudip Sir

2

PHP Basics: PHP syntax, variables, data types, operators, and control structures. Functions in PHP: Defining and using functions, function scope, including files, and built-in functions
Arrays and Strings: Working with arrays, array functions, string manipulation, and string functions

Basic PHP Syntax

- PHP code is written inside `<?php ... ?>` tags. ✓
- Statements end with a **semicolon (;)**. ✓
- PHP is **case-insensitive** for keywords (echo, if, etc.)
but **case-sensitive** for variable names. 

```
<!DOCTYPE html>
<html>
<body>

<h1>My first PHP page</h1>

<?php
echo 'Hello World!';
?>

</body>
</html>
```

PHP Statements

A **statement** is an instruction given to PHP.

```
<?php
$name = "John";    // Assignment statement
echo $name;        // Output statement
?>
```

echo

echo is used to output data.

```
<?php
```

```
echo "Hello Students"; ✓
```

```
?>
```

Output:

Hello Students

echo "\$name"

Comments in PHP

PHP supports **two types of comments**:

1. Single-line comments

```
<?php
// This is a single-line comment using double slashes
echo "Hello World!"; // Output greeting

# This is also a single-line comment using hash
echo "PHP Comments Example";
?>
```

//
#

2. Multi-line comments

```
<?php
/*
   This is a multi-line comment.
   It can span across multiple lines.
   Useful for detailed explanations.
*/
echo "Learning PHP comments!";
?>
```

/*

*/

PHP is Case-Sensitive

PHP variable names are **case-sensitive**.

Example:

\$name = "Rahul";

and:

\$Name = "Priya";

are two different variables.

echo \$name; ✓

echo \$Name; ✓

can produce different results.

PHP Variables

A variable in PHP is a container used to store data such as numbers, strings, arrays, or objects.

The value stored in a variable can be changed or updated during the execution of the script.

- All variable names start with a dollar sign (\$).
- Variables can store different data types, like integers, strings, arrays, etc.
- PHP is loosely typed, so you don't need to declare a data type explicitly.
- Variable values can change during the script's execution.

Syntax: ✓

```
$variable_name = value;
```

Example:

```
<?php
$name = "XYZ"; // String
$age = 30;      // Integer
$salary = 45000.50; // Float
$isEmployed = true; // Boolean
?>
```

Example of Valid and Invalid Variable Names:

```
<?php
$firstName = "Alice"; // Valid
$_age = 25;           // Valid

$2ndPlace = "Bob";    // Invalid: Cannot start with a number
$class = "Physics";   // Valid, but avoid reserved words
?>
```


Variable Scope in PHP

PHP has **three main scopes**:

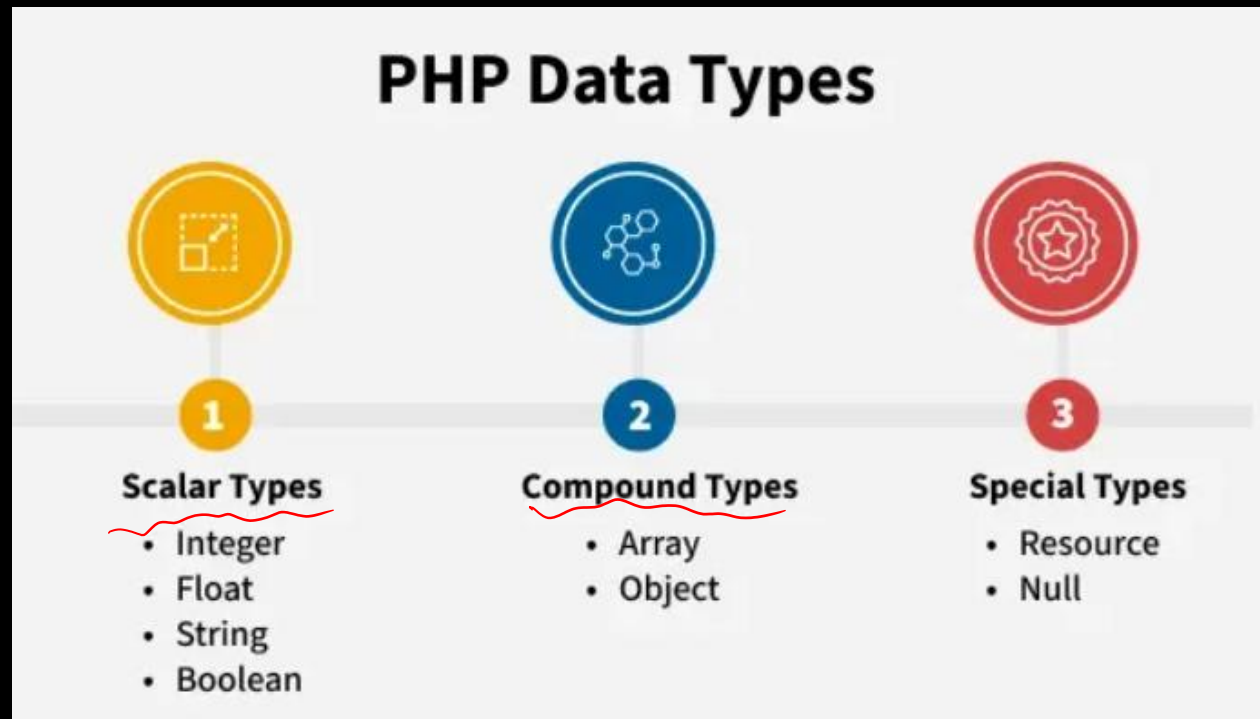
1. **Local** – inside a function.
2. **Global** – declared outside functions.
3. **Static** – retains value between function calls.

```
<?php
$globalVar = "I am global";

function testScope() {
    global $globalVar; // Access global variable
    echo $globalVar;
}
testScope();
?>
```

DATA TYPES

In PHP, data types define the kind of value a variable can hold. PHP is a loosely typed language, meaning you don't need to declare the data type of a variable. It is automatically assigned based on the value.



1. Integer

Whole numbers (positive, negative, or zero).

Example:

```
$age = 25; // integer
```

2. Float / Double

Numbers with decimal points or in exponential form.

Example:

```
$price = 99.99; // float
```

3. String

Sequence of characters enclosed in quotes.

Example:

```
$name = "John Doe"; // string
```

4. Boolean

Represents true or false.

Example:

```
$isActive = true; // boolean
```

✓ 1. Array

Collection of values (indexed or associative).

Example:

```
$colors = ["red", "green", "blue"]; // indexed array  
$person = ["name" => "Alice", "age" => 30]; // associative array
```

✓ 2. Object

Instance of a class containing properties and methods.

Example:

```
class Car {  
    public $brand;  
    function __construct($brand) {  
        $this->brand = $brand;  
    }  
}  
$car = new Car("Toyota");
```

1. NULL

Represents a variable with no value.

Example:

```
$x = null;
```

2. Resource

Holds a reference to an external resource (e.g., database connection, file handle).

Example:

```
$file = fopen("test.txt", "r");
```

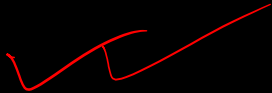
PHP Operators

In PHP, operators are special symbols used to perform operations on variables and values. Operators help you perform a variety of tasks, such as mathematical calculations, string manipulations, logical comparisons, and more.

1. Arithmetic Operators

Arithmetic operators are used to perform basic arithmetic operations like addition, subtraction, multiplication, division, and modulus.

Operator	Name	Syntax	Operation
+	Addition	$\$x + \y	Sum the operands
-	Subtraction	$\$x - \y	Differences in the Operands
*	Multiplication	$\$x * \y	Product of the operands
/	Division	$\$x / \y	The quotient of the operands
**	Exponentiation	$\$x ** \y	$\$x$ raised to the power $\$y$
%	Modulus	$\$x \% \y	The remainder of the operands




```
<?php
```

```
// Define two numbers
```

```
$x = 10;
```

```
$y = 3;
```

```
// Addition
```

```
echo "Addition: " . ($x + $y) . "\n";
```

```
// Subtraction
```

```
echo "Subtraction: " . ($x - $y) . "\n";
```

```
// Multiplication
```

```
echo "Multiplication: " . ($x * $y) . "\n";
```

```
// Division
```

```
echo "Division: " . ($x / $y) . "\n";
```

```
// Exponentiation
```

```
echo "Exponentiation: " . ($x ** $y) . "\n";
```

```
// Modulus
```

```
echo "Modulus: " . ($x % $y) . "\n";
```

```
?>
```

"A is", \$a, "and B is"
\$b."
"

Output

Addition: 13

Subtraction: 7

Multiplication: 30

Division: 3.3333333333333

Exponentiation: 1000

Modulus: 1

2. Logical Operators

Logical operators are used to operate with conditional statements. These operators evaluate conditions and return a boolean result (true or false).

Operator	Name	Syntax	Operation
and	Logical AND	\$x and \$y	True if both the operands are true else false
or	Logical OR	\$x or \$y	True if either of the operands is true otherwise, it is false
xor	Logical XOR	\$x xor \$y	True if either of the operands is true and false if both are true
&&	Logical AND	\$x && \$y	True if both the operands are true else false
	Logical OR	\$x \$y	True if either of the operands is true otherwise, it is false
!	Logical NOT	!\$x	True if \$x is false

```
<?php
$x = 50;
$y = 30;
    if ($x == 50 and $y == 30)
        echo "and Success \n";

    if ($x == 50 or $y == 20)
        echo "or Success \n";

    if ($x == 50 xor $y == 20)
        echo "xor Success \n";

    if ($x == 50 && $y == 30)
        echo "&& Success \n";

    if ($x == 50 || $y == 20)
        echo "|| Success \n";

    if (!$z)
        echo "! Success \n";
?>
```

Output

```
and Success
or Success
xor Success
&& Success
|| Success
! Success
```

3. Comparison Operators

Comparison operators are used to compare two values and return a Boolean result (true or false).

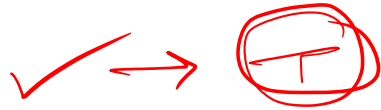
Operator	Name	Syntax	Operation
==	Equal To	\$x == \$y	Returns True if both the operands are equal
!=	Not Equal To	\$x != \$y	Returns True if both the operands are not equal
✓ <>	Not Equal To	<u>\$x <> \$y</u>	Returns True if both the operands are unequal
✓ ===	Identical	\$x === \$y	Returns True if both the operands are equal and are of the same type
!==	Not Identical	<u>\$x !== \$y</u>	Returns True if both the operands are unequal and are of different types
<	Less Than	\$x < \$y	Returns True if \$x is less than \$y
>	Greater Than	\$x > \$y	Returns True if \$x is greater than \$y
<=	Less Than or Equal To	\$x <= \$y	Returns True if \$x is less than or equal to \$y
>=	Greater Than or Equal To	\$x >= \$y	Returns True if \$x is greater than or equal to \$y

```
<?php
```

```
$a = 80;  
$b = 50;  
$c = "80";
```

```
// Here var_dump function has been used to  
// display structured information. We will learn  
// about this function in complete details in further  
// articles.
```

```
var_dump($a == $c) . "\n";
```



```
var_dump($a != $b) . "\n";
```

```
var_dump($a <> $b) . "\n";
```

```
var_dump($a === $c) . "\n";
```



```
var_dump($a !== $c) . "\n";
```

```
var_dump($a < $b) . "\n";
```

```
var_dump($a > $b) . "\n";
```

```
var_dump($a <= $b) . "\n";
```

```
var_dump($a >= $b);
```

```
?>
```

4. Conditional or Ternary Operators

These operators are used to compare two values and take either of the results simultaneously, depending on whether the outcome is TRUE or FALSE. These are also used as a shorthand notation for the *if...else* statement that we will read in the article on decision making.

Syntax:

`$var = (condition)? value1 : value2;`

Operator	Name	Operation
?:	Ternary	If the condition is true? then \$x : or else \$y. This means that if the condition is true, then the left result of the colon is accepted otherwise, the result is on the right.

```
<?php
```

```
    $x = -12;
```

```
    echo ($x > 0) ? 'The number is positive' : 'The number is negative';  
?>
```

Output

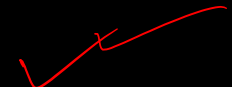
The number is negative



5. Assignment Operators

Assignment operators are used to assign values to variables. These operators allow you to assign a value and perform operations in a single step.

Operator	Name	Syntax	Operation
=	Assign	$\$x = \y	Operand on the left obtains the value of the operand on the right
+=	Add then Assign	$\$x += \y	Simple Addition same as $\$x = \$x + \$y$
-=	Subtract then Assign	$\$x -= \y	Simple subtraction same as $\$x = \$x - \$y$
*=	Multiply then Assign	$\$x *= \y	Simple product same as $\$x = \$x * \$y$
/=	Divide, then assign (quotient)	$\$x /= \y	Simple division same as $\$x = \$x / \$y$
%=	Divide, then assign (remainder)	$\$x \quad \% =$ $\$y$	Simple division same as $\$x = \$x \% \$y$




```
<?php
```

```
// Simple assign operator
```

```
$y = 75;  
echo $y, "\n";
```

```
// Add then assign operator
```

```
$y = 100;  
$y += 200;  
echo $y, "\n";
```

```
// Subtract then assign operator
```

```
$y = 70;  
$y -= 10;  
echo $y, "\n";
```

```
// Multiply then assign operator
```

```
$y = 30;  
$y *= 20;  
echo $y, "\n";
```

```
// Divide then assign(quotient) operator
```

```
$y = 100;  
$y /= 5;  
echo $y, "\n";
```

```
// Divide then assign(remainder) operator
```

```
$y = 50;  
$y %= 5;  
echo $y;
```

```
?>
```

Output

```
75  
300  
60  
600  
20  
0
```

6. Array Operators

These operators are used in the case of arrays. Here are the array operators, along with their syntax and operations, that PHP provides for the array operation.

Operator	Name	Syntax	Operation
+	Union	$\$x + \y	Union of both, i.e., $\$x$ and $\$y$
==	Equality	$\$x == \y	Returns true if both have the same key-value pair
!=	Inequality	$\$x != \y	Returns True if both are unequal
===	Identity	$\$x === \y	Returns True if both have the same key-value pair in the same order and of the same type
!==	Non-Identity	$\$x !== \y	Returns True if both are not identical to each other
<>	Inequality	$\$x <> \y	Returns True if both are unequal

```
<?php
    $x = array("k" => "Car", "l" => "Bike");
    $y = array("a" => "Train", "b" => "Plane");

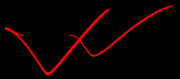
    var_dump($x + $y);
    var_dump($x == $y) . "\n";
    var_dump($x != $y) . "\n";
    var_dump($x <> $y) . "\n";
    var_dump($x === $y) . "\n";
    var_dump($x !== $y) . "\n";
?>
```

Output:

```
array(4) {
    ["k"]=>
    string(3) "Car"
    ["l"]=>
    string(4) "Bike"
    ["a"]=>
    string(5) "Train"
    ["b"]=>
    string(5) "Plane"
}
bool(false)
bool(true)
bool(true)
bool(false)
bool(true)
```

7. Increment/Decrement Operators

These are called the unary operators as they work on single operands. These are used to increment or decrement values.



Operator	Name	Syntax	Operation
++	Pre-Increment	++\$x	First, increment \$x by one, then return \$x
--	Pre-Decrement	--\$x	First, decrement \$x by one, then return \$x
++	Post-Increment	\$x++	First returns \$x, then increment it by one
--	Post-Decrement	\$x--	First, return \$x, then decrement it by one

<?php

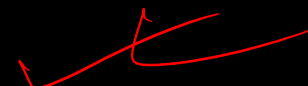
```
$x = 2;  
echo ++$x, " First increments then prints \n";  
echo $x, "\n";
```

```
$x = 2;  
echo $x++, " First prints then increments \n";  
echo $x, "\n";
```

```
$x = 2;  
echo --$x, " First decrements then prints \n";  
echo $x, "\n";
```

```
$x = 2;  
echo $x--, " First prints then decrements \n";  
echo $x;
```

?>



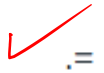
Output

```
3 First increments then prints  
3  
2 First prints then increments  
3  
1 First decrements then prints  
1  
2 First prints then decrements  
1
```

8. String Operators

This operator is used for the concatenation of 2 or more strings using the concatenation operator ('.').

We can also use the concatenating assignment operator ('.=') to append the argument on the right side to the argument on the left side.

Operator	Name	Syntax	Operation
 .	Concatenation	<code>\$x.\$y</code>	Concatenated \$x and \$y
 .=	Concatenation and assignment	<code>\$x.=\$y</code>	First, it concatenates then assigns, the same as <code>\$x = \$x.\$y</code>

```
<?php
    $x = "Great";
    $y = "Heights";
    $z = "Academy!!!";
    echo $x . $y . $z, "\n";
    $x .= $y . $z; ✓
    echo $x;
?>
```

```
GreatHeightsAcademy!!!
GreatHeightsAcademy!!!
```

$$a + = b \quad \Rightarrow \quad a = a + b$$

CONTROL STRUCTURES

Control structures determine **which code runs and how many times it runs.**

They include:

Decision-> if, if-else, if-elseif-else, switch

Repetition-> for, while, do-while, foreach

1. if Statement

The if statement is the simplest form of decision making. It executes a block of code if the specified condition evaluates to true. If the condition evaluates to false, the block of code is skipped.

```
if (condition){  
    // if TRUE then execute this code  
}
```

```
<?php  
$x = 12;  
  
if ($x > 0) {  
    echo "The number is positive";  
}  
?>
```

2. if...else Statement

The if-else statement is an extension of the if statement. It allows you to specify an alternative block of code that is executed when the condition is false. This is useful when there are two mutually exclusive conditions.

```
if (condition) {  
    // if TRUE then execute this code  
}  
else{  
    // if FALSE then execute this code  
}
```

```
<?php  
$x = -12;  
if ($x > 0) {  
    echo "The number is positive";  
}  
else{  
    echo "The number is negative";  
}  
?>
```

3. if...elseif...else Statement

In scenarios where you need to evaluate multiple conditions, you can use the if-elseif-else ladder. This construct allows you to check multiple conditions in sequence. The first condition that evaluates to true will execute its corresponding block of code, and all other conditions will be skipped.

```
<?php
$x = "August";

if ($x == "January") {
    echo "Happy Republic Day";
}
elseif ($x == "August") {
    echo "Happy Independence Day!!!";
}
else{
    echo "Nothing to show";
}
?>
```

```
if (condition) {
    // if TRUE then execute this code
}
elseif {
    // if TRUE then execute this code
}
elseif {
    // if TRUE then execute this code
}
else {
    // if FALSE then execute this code
}
```

4. switch Statement

The "switch" performs in various cases i.e., it has various cases to which it matches the condition and appropriately executes a particular case block. It first evaluates an expression and then compares with the values of each case. If a case matches then the same case is executed. To use switch, we need to get familiar with two different keywords namely, **break** and **default**.

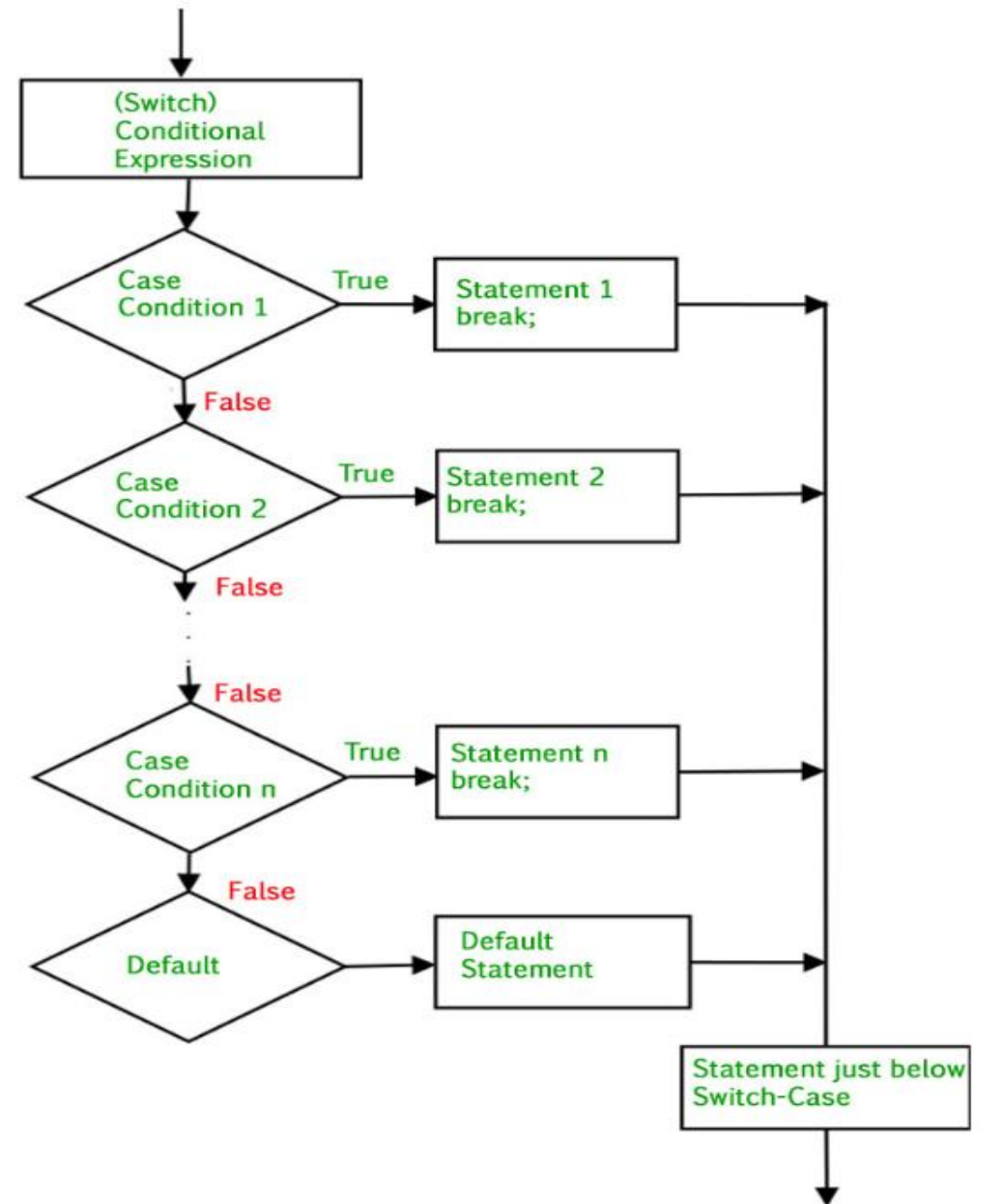
- The **break** statement is used to stop the automatic control flow into the next cases and exit from the switch case.
- The **default** statement contains the code that would execute if none of the cases match.

```
switch (variable) {  
    case value1:  
        // Code to be executed if the variable equals value1  
        break;  
    case value2:  
        // Code to be executed if the variable equals value2  
        break;  
    default:  
        // Code to be executed if no cases match  
}
```

```

<?php
$day = "Monday";
switch ($day) {
    case "Monday":
        echo "Start of the week!";
        break;
    case "Friday":
        echo "End of the week!";
        break;
    case "Saturday":
    case "Sunday":
        echo "Weekend!";
        break;
    default:
        echo "Mid-week!";
}
?>

```



PHP Loops

In PHP, Loops are used to repeat a block of code multiple times based on a given condition. PHP provides several types of loops to handle different scenarios, including while loops, for loops, do...while loops, and foreach loops.

Types of Loops in PHP

Below are the following types of loops in PHP:

- for loop
- while loop
- do-while loop
- foreach loop

1. PHP for Loop

PHP for loop is used when you know exactly how many times you want to iterate through a block of code. It consists of three expressions:

- ✓ **Initialization:** Sets the initial value of the loop variable.
- ✓ **Condition:** Checks if the loop should continue.
- ✓ **Increment/Decrement:** Changes the loop variable after each iteration.

```
for ( Initialization; Condition; Increment/Decrement ) {  
    // Code to be executed  
}
```

Example: Printing numbers from 1 to 5 using a for loop.

```
<?php  
  
// Code to illustrate for loop  
for ($num = 1; $num <= 5; $num += 1) {  
    echo $num . "\n";  
}  
  
?>
```

2. PHP while Loop

The while loop is also an entry control loop like for loops. It first checks the condition at the start of the loop, and if it's true then it enters into the loop and executes the block of statements and goes on executing it as long as the condition holds true.

```
while ( condition ) {  
    // Code is executed  
}
```

Example: Printing numbers from 1 to 5.

```
<?php  
$num = 1;  
while ($num <= 5) {  
    echo $num . "\n";  
    $num++;  
}  
?>
```


3. PHP do-while Loop

The do-while loop is an exit control loop, which means, it first enters the loop, executes the statements, and then checks the condition. Therefore, a statement is executed at least once using the do...while loop. After executing once, the program is executed as long as the condition holds true.

```
do {  
    // Code is executed  
} while ( condition );
```

Example: Printing numbers from 1 to 5.

```
<?php  
$num = 1;  
do {  
    echo $num . "\n";  
    $num++;  
} while ($num <= 5);  
?>
```

4. PHP foreach Loop

This foreach loop is used to iterate over arrays. For every counter of loop, an array element is assigned, and the next counter is shifted to the next element. It simplifies working with arrays and objects by automatically iterating through each element.

```
foreach ( $array as $value ) {  
    // Code to be executed  
}  
or  
foreach ( $array as $key => $value ) {  
    // Code to be executed  
}
```

Example: Iterating through an array

```
<?php  
// foreach loop over an array  
$arr = array (10, 20, 30, 40, 50, 60);  
foreach ($arr as $val) {  
    echo $val . " ";  
}  
echo "\n";  
// foreach loop over an array with keys  
$ages = array(  
    "Anjali" => 25,  
    "Kriti" => 30,  
    "Ayushi" => 22  
);  
foreach ($ages as $name => $age) {  
    echo $name . " => " . $age . "\n";  
}  
?>
```

Output

```
10 20 30 40 50 60  
Anjali => 25  
Kriti => 30  
Ayushi => 22
```

PHP Functions

A function in PHP is a self-contained block of code that performs a specific task. It can accept inputs (parameters), execute a set of statements, and optionally return a value.

- PHP functions allow code reusability by encapsulating a block of code to perform specific tasks.
- Functions can accept parameters and return values, enabling dynamic behavior based on inputs.
- PHP supports both built-in functions and user-defined functions, enhancing flexibility and modularity in code.

```
<?php
```

```
// Creating a function that adds two numbers
```

```
function addNumbers($a, $b) {
```

```
    // code to be executed
```

```
    $result = $a + $b;
```

```
    // returning the sum
```

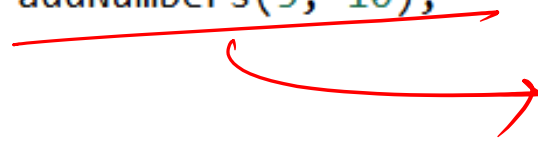
```
    return $result;
```

```
}
```

```
// Calling the function
```

```
echo addNumbers(5, 10);
```

```
?>
```



Syntax

```
function functionName($param1, $param2) {  
    // Code to be executed  
    return $result; // optional  
}
```

Calling a Function in PHP

Once a function is declared, it can be invoked (called) by simply using the function name followed by parentheses.

```
<?php
    function sum($a, $b) {
        return $a + $b;
    }
    echo sum(5, 3);    // Outputs: 8
?>
```

Output

8

- The **function sum()** is defined to take two parameters, \$a and \$b, and return their sum.
- The function is called with 5 and 3 as arguments, and it returns the result, which is 8.

Types of Functions in PHP

PHP has two types of functions:

1. User-Defined Functions

A user-defined function is created to perform a specific task as per the developer's need. These functions can accept parameters, perform computations, and return results.

- The function `addNumbers` is defined by the user to take two parameters, `$a` and `$b`, and returns their sum (`$a + $b`).
- The function is called with 5 and 3 as arguments, so it adds these numbers together.

```
<?php
function addNumbers($a, $b) {
    return $a + $b;
}
echo addNumbers(5, 3); // Output: 8
?>
```

Output

8

2. Built-in Functions

PHP comes with many built-in functions that can be directly used in your code. For example, `strlen()`, `substr()`, `array_merge()`, `date()` and etc are built-in PHP functions. These functions provide useful functionalities, such as string manipulation, date handling, and array operations, without the need to write complex logic from scratch.

- A string variable `$str` is defined with the value "Hello, World!".
- The `strlen()` function is the built-in function in PHP that is used to calculate the length of the string `$str`.

```
<?php
    $str = "Hello, World!";
    echo strlen($str); // Output: 13
?>
```

Output

13

PHP Function Parameters and Arguments

Functions can accept input values, known as parameters or arguments.

- Parameters are variables defined in the function declaration that accept values.
- Arguments are the actual values passed to the function when it is called.

These values can be passed to the function in two ways:

1. Passing by Value

When you pass a value by value, the function works with a copy of the argument, so the original value remains unchanged.

- The function `add()` is defined with parameters `$x` and `$y`.
- The function adds `$x` and `$y` and returns the sum.
- The function is called with values 2 and 3, and the result 5 is printed.

```
<?php
function add($x, $y) {
    $x = $x + $y;
    return $x;
}

echo add(2, 3); // Outputs: 5
?>
```

Output

5

2. Passing by Reference

By passing an argument by reference, any changes made inside the function will affect the original variable outside the function.

- The function `addByRef()` is defined with `$x` passed by reference.
- The function modifies `$x` directly, adding `$y` to it.
- After calling the function with `$a = 5` and `3`, the value of `$a` becomes `8`.



```
<?php
function addByRef(&$x, $y) {
    $x = $x + $y;
}
$a = 5;
addByRef($a, 3);
echo $a; // Outputs: 8
?>
```

Output

8